

Online Bookstore & Library Management System

Data Engineering Capstone Project — Medallion Pipeline

Author: Shrim Sethi

Internship: Celebal Technology — Data Engineering Track

Database: MySQL 8 (celebal)

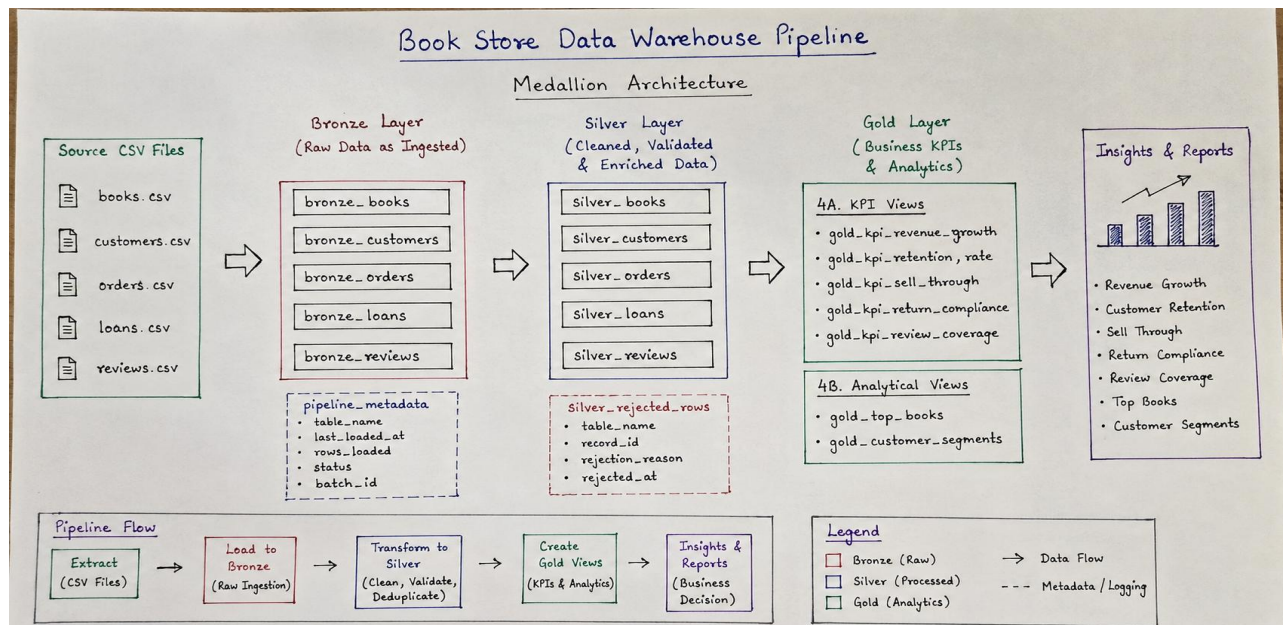
Architecture: Medallion (Bronze → Silver → Gold)

1. What this project is about

This project is about building a full data pipeline for a company that runs both an online bookstore and a public library. The company has raw data sitting in 5 operational tables (books, customers, orders, loans, reviews) and the goal is to turn that raw data into clean, trusted, business-ready data that an executive dashboard can use.

To do this, I used the Medallion Architecture, which breaks the pipeline into 3 layers:

Layer	What it holds	Purpose
Bronze	Raw data, exact copy of source	Source of truth, nothing changed, nothing deleted
Silver	Cleaned and validated data	Trusted data, duplicates removed, bad rows rejected
Gold	Business-ready aggregates and KPIs	What the dashboard actually reads from



2. Raw Data (Source Tables)

Table	Rows	Description
books	190	Book catalog with price, stock, genre
customers	2,800	Customer master data with membership type
orders	28,336	Purchase transactions
loans	9,614	Library borrow/return records
reviews	5,621	Customer ratings and review text

3. Bronze Layer — Raw Ingestion

3.1 What it does

The bronze layer is just a copy of the source tables, nothing is cleaned here. Every bronze table has two extra columns added on top of the source structure:

Column	Type	Why
ingested_at	DATETIME	When this row was loaded
batch_id	VARCHAR	Which load run this row came from

3.2 Incremental Load (Watermark Pattern)

Instead of loading all data every time, I used a watermark pattern. A table called pipeline_metadata keeps track of the last successful load time for every bronze table.

table_name	last_loaded_at	rows_loaded	status
bronze_books	last run time	count	SUCCESS
bronze_customers	last run time	count	SUCCESS
bronze_orders	last run time	count	SUCCESS
bronze_loans	last run time	count	SUCCESS
bronze_reviews	last run time	count	SUCCESS

Every load reads this watermark first, loads only new rows (using order_date, joined_on, loan_date, created_at as the date to compare), then updates the watermark once the load succeeds. This whole insert + update is wrapped in a transaction, so if the insert fails halfway, the watermark does not move.

3.3 Watermark column used per table

Table	Watermark column	Reason
bronze_books	none (books don't have a load-relevant date)	uses row-existence check instead
bronze_customers	joined_on	new customer sign-ups

Table	Watermark column	Reason
bronze_orders	order_date	new purchase activity
bronze_loans	loan_date	new borrow activity
bronze_reviews	created_at	new review activity

4. Silver Layer — Cleaning & Validation

4.1 What it does

The silver layer reads from bronze and produces a clean version of each table. Three things happen here:

- **Deduplicate** — if the same primary key appears more than once in bronze, only the latest one (by ingested_at) is kept, using ROW_NUMBER().
- **Validate** — every row is checked against a set of quality rules. Rows that fail go into silver_rejected_rows with a reason, rows that pass go into the silver table.
- **Enrich** — some columns are calculated fresh, like order_value for orders and days_overdue / overdue_category for loans.

4.2 Quality rules used per table

Table	Rule checked	Rejection reason logged
books	title not empty, author not empty, price > 0, stock >= 0, published_on not null	Missing Title / Missing Author / Invalid Price / Invalid Stock / Missing Published Date
customers	name not empty, email format valid, joined_on not null, membership not empty	Missing Name / Invalid Email / Missing Join Date / Missing Membership
orders	quantity > 0, order_date not null, status not empty	Invalid Quantity / Missing Order Date / Missing Status
loans	loan_date not null, due_date not null, due_date >= loan_date	Missing Loan Date / Missing Due Date / Invalid Due Date
reviews	rating between 1 and 5	Invalid Rating

```
1 • SELECT * FROM celebal.silver_rejected_rows;
```

table_name	source_id	rejection_reason	rejected_at
bronze_customers	70	Invalid Email	2026-07-19 01:34:24
bronze_customers	130	Invalid Email	2026-07-19 01:34:24
bronze_customers	354	Invalid Email	2026-07-19 01:34:24
bronze_customers	402	Invalid Email	2026-07-19 01:34:24
bronze_customers	494	Invalid Email	2026-07-19 01:34:24
bronze_customers	497	Missing Membership	2026-07-19 01:34:24
bronze_customers	673	Missing Membership	2026-07-19 01:34:24
bronze_customers	711	Missing Membership	2026-07-19 01:34:24
bronze_customers	714	Invalid Email	2026-07-19 01:34:24
bronze_customers	763	Invalid Email	2026-07-19 01:34:24
bronze_customers	778	Missing Membership	2026-07-19 01:34:24
bronze_customers	966	Invalid Email	2026-07-19 01:34:24
bronze_customers	1038	Invalid Email	2026-07-19 01:34:24
bronze_customers	1097	Invalid Email	2026-07-19 01:34:24
bronze_customers	1116	Invalid Email	2026-07-19 01:34:24
bronze_customers	1235	Invalid Email	2026-07-19 01:34:24
bronze_customers	1466	Invalid Email	2026-07-19 01:34:24
bronze_customers	1528	Invalid Email	2026-07-19 01:34:24
bronze_customers	1588	Invalid Email	2026-07-19 01:34:24
bronze_customers	2052	Missing Membership	2026-07-19 01:34:24

4.3 Derived / enriched columns

Table	New column	How it is calculated
orders	order_value	quantity × book price
loans	days_overdue	days between due_date and return_date (or today, if not yet returned), never negative
loans	overdue_category	ON TIME (0 days) / MILD (1–5 days) / SEVERE (6–15 days) / CRITICAL (15+ days)

4.4 A bug I ran into and fixed

For reviews, I initially added extra checks (review text not empty, created_at not null) that were not part of the required rules. This pushed the rejection rate for reviews up to 9.7%, which failed the < 5% healthy target. I removed the extra checks and kept only the one rule that was actually required (rating between 1 and 5). After this fix, the rejection rate for reviews dropped to about 1.76%.

Lesson learned: don't add validation rules that were not asked for, it only creates false rejections and makes the data quality numbers look worse than they actually are.

4.5 Accepted vs Rejected summary

Table	Bronze rows	Accepted (Silver)	Rejected	Rejection %
bronze_books	190	190	0	0.00%
bronze_customers	2,800	2,771	29	1.04%
bronze_orders	28,336	27,953	217	0.77%
bronze_loans	9,614	9,538	76	0.79%
bronze_reviews	5,621	5,522	99	1.76%

1 • `SELECT * FROM celebal.gold_pipeline_health;`

Result Grid   Filter Rows: Export:  Wrap Cell Content: 

	table_name	last_loaded_at	rows_in_bronze	rows_in_silver	rows_rejected	rejection_rate_pct	pipeline_status
▶	bronze_books	2026-07-19 01:08:12	190	190	0	0.00	HEALTHY
	bronze_customers	2026-07-19 01:09:15	2800	2771	29	1.04	HEALTHY
	bronze_loans	2026-07-19 01:16:40	9614	9538	76	0.79	HEALTHY
	bronze_orders	2026-07-19 01:09:47	28336	27953	217	0.77	HEALTHY
	bronze_reviews	2026-07-19 01:17:22	5621	5076	99	1.76	HEALTHY

5. Gold Layer — KPI Views & Business Aggregates

The gold layer only reads from silver tables, never from bronze or the raw source. It has two types of views:

5.1 KPI Views (5 required)

Every KPI view returns the same shape of output: current value, target value, PASS/FAIL status, and when it was calculated.

KPI View	What it measures	Target
gold_kpi_revenue_growth	Month-over-month % change in DELIVERED order revenue	$\geq 5\%$ in any 3 consecutive months
gold_kpi_retention_rate	% of customers who ordered in 2 consecutive calendar months	$\geq 60\%$
gold_kpi_sell_through	% of books with at least 1 DELIVERED order	$\geq 70\%$
gold_kpi_return_compliance	% of loans returned on or before due_date	$\geq 75\%$
gold_kpi_review_coverage	% of DELIVERED orders that have a matching review	$\geq 40\%$

How each one works, in simple words:

- **Revenue Growth** — Add up delivered order revenue for each month, then compare it with the previous month using the LAG() window function to get the % change.
- **Retention Rate** — Find which customers ordered in month X, then check if the same customer also ordered in month X+1 (self join on customer + next month).
- **Sell-Through** — Count how many distinct books had at least 1 delivered order, divide by the total number of books.
- **Return Compliance** — Count how many loans were returned on or before the due date, divide by total loans.
- **Review Coverage** — For every delivered order, check if a review exists for the same customer + book combination (orders and reviews don't share an order id, so this is matched on customer + book).

```
1 • SELECT * FROM celebal.gold_kpi_retention_rate;
```

Result Grid

Filter Rows:

Export:

Wrap Cell Content:

	kpi_value	kpi_target	status	calculated_at
▶	67.11	60	PASS	2026-07-19 10:39:29

5.2 Analytical Views (2 required)

View	What it shows
gold_top_books	Top 10 books by revenue within each genre, with total units sold and average rating
gold_customer_segments	Every customer classified as HIGH VALUE (> ₹20,000), MID VALUE (₹5,000–₹20,000), or LOW VALUE (< ₹5,000), based on total spend on delivered orders. Customers with zero spend are also included using a LEFT JOIN

```
1 • SELECT * FROM celebal.gold_customer_segments;
```

Result Grid				
Filter Rows:		Export:	Wrap Cell Content:	Fetch rows:
customer_id	customer_name	total_spend	customer_segment	
1	Aadhya Chauhan	183693.93	HIGH VALUE	
2	Himani Tripathi	1619.37	LOW VALUE	
3	Tushar Raman	13584.59	MID VALUE	
4	Pooja Banerjee	10187.97	MID VALUE	
5	Shreya Krishnan	924.95	LOW VALUE	
6	Sunita Menon	2455.30	LOW VALUE	
7	Rudra Chakraborty	13696.24	MID VALUE	
8	Nitin Trivedi	842.04	LOW VALUE	
9	Grish Bhatia	11984.59	MID VALUE	
10	Shubham Iyer	25871.42	HIGH VALUE	
11	Shaurya Mishra	1309.66	LOW VALUE	
12	Anil Patil	1080.57	LOW VALUE	
13	Tushar Malhotra	3242.84	LOW VALUE	
14	Bhavna Roy	10357.21	MID VALUE	
15	Rudra Khanna	8441.97	MID VALUE	
16	Prasad Chakraborty	9532.89	MID VALUE	
17	Manish Khanna	15348.09	MID VALUE	
18	Sai Kulkarni	4734.89	LOW VALUE	
19	Nisha Ghosh	10167.11	MID VALUE	
20	Sai Pandev	3742.54	LOW VALUE	

6. Pipeline Health & Audit

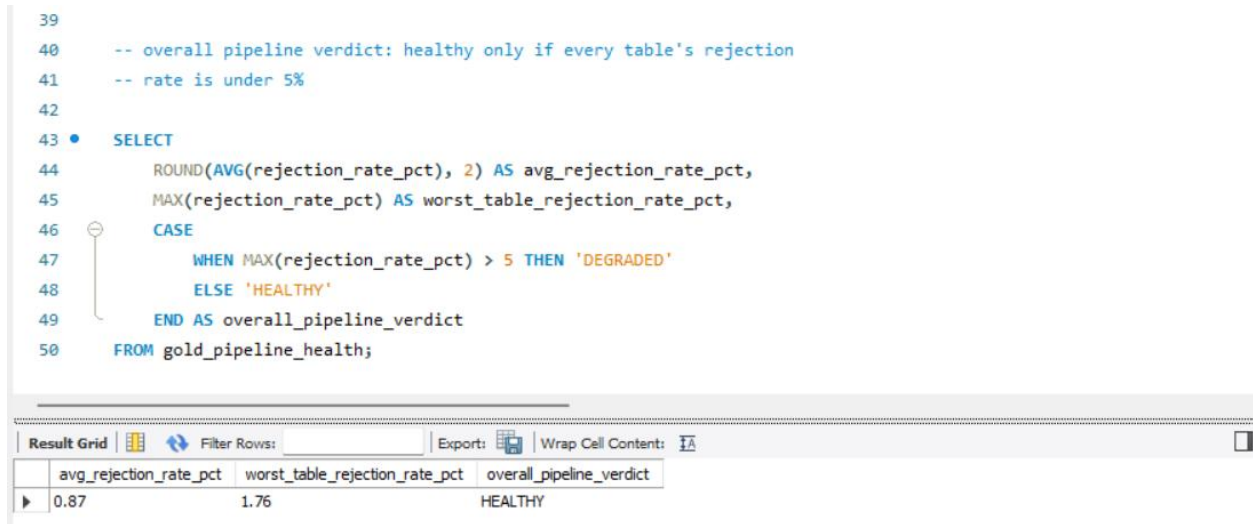
A view called gold_pipeline_health was built to check the health of the whole pipeline in one query. It joins bronze row counts, silver row counts, and rejected row counts for every table, and calculates a rejection rate.

Rule: if rejection rate for any table is above 5%, that table's status shows DEGRADED, otherwise HEALTHY.

table_name	rows_in_bronze	rows_in_silver	rows_rejected	rejection_rate_pct	pipeline_status
bronze_books	190	190	0	0.00	HEALTHY
bronze_customers	2,800	2,771	29	1.04	HEALTHY
bronze_orders	28,336	27,953	217	0.77	HEALTHY
bronze_loans	9,614	9,538	76	0.79	HEALTHY
bronze_reviews	5,621	5,522	99	1.76	HEALTHY

Overall pipeline verdict: HEALTHY (worst table rejection rate is 1.76%, well under the 5% limit)

```
39
40 -- overall pipeline verdict: healthy only if every table's rejection
41 -- rate is under 5%
42
43 • SELECT
44     ROUND(AVG(rejection_rate_pct), 2) AS avg_rejection_rate_pct,
45     MAX(rejection_rate_pct) AS worst_table_rejection_rate_pct,
46     CASE
47         WHEN MAX(rejection_rate_pct) > 5 THEN 'DEGRADED'
48         ELSE 'HEALTHY'
49     END AS overall_pipeline_verdict
50 FROM gold_pipeline_health;
```



The screenshot shows a SQL query in a code editor. The query calculates the average and maximum rejection rates from a table named 'gold_pipeline_health' and returns a verdict of 'HEALTHY' because the maximum rejection rate (1.76%) is less than 5%. Below the code, a 'Result Grid' is visible with three columns: 'avg_rejection_rate_pct', 'worst_table_rejection_rate_pct', and 'overall_pipeline_verdict'. The first row of data shows values 0.87, 1.76, and HEALTHY.

avg_rejection_rate_pct	worst_table_rejection_rate_pct	overall_pipeline_verdict
0.87	1.76	HEALTHY

7. Tech Stack Used

Tool	Purpose
MySQL 8	Database engine, all pipeline logic
Window Functions (ROW_NUMBER, LAG)	Deduplication and month-over-month comparison
CTEs (WITH clause)	Breaking complex KPI logic into readable steps
Transactions	Making bronze load safe and atomic
Views	All silver-to-gold business logic, kept query-ready

8. Final Folder Structure

```
Celebal_Final_Project/
├── raw_data/
│   ├── books.csv
│   ├── customers.csv
│   ├── loans.csv
│   ├── orders.csv
│   └── reviews.csv
├── sql/
│   ├── task1_schema.sql
│   ├── task2_bronze_load.sql
│   ├── task3_silver_transform.sql
│   ├── task4_gold_views.sql
│   └── task5_audit.sql
├── screenshots/
│   ├── Bronze_layer/
│   ├── Silver_layer/
│   ├── Gold_kpi/
│   └── Health_pipeline/
└── Readme.md
```

9. Conclusion

The pipeline moves data through 3 layers, raw copy in bronze, cleaned and validated in silver, business-ready KPIs in gold. All five KPI targets are being tracked with clear PASS/FAIL logic, and the pipeline health check shows the overall data quality is well within the accepted 5% rejection limit. The main learning from this project was how small mistakes, like a missing existence check or an extra validation rule, can silently break numbers further down the pipeline, and how important it is to trace rejections back to their exact cause instead of just accepting the final count.